

LLM HANDBOOK

50 Must-Know Concepts

1. WHAT is LLM?

★ What is LLM?

Large Language Model = Deep Learning + Massive Text + Billions Parameters + Predicts Next Token.

- Large Language Model is deep learning model trained on massive text data — billions parameters — predicts next token.

★ What is Examples?

- GPT-4 · OpenAI
- Claude · Anthropic
- Gemini · Google
- Llama · Meta · open source
- Mistral

★ What is Why LLMs?

Emergent abilities · in-context learning, zero-shot · few-shot reasoning.

★ What is LLM Scale?

- Parameters: 7B · 13B · 70B · 175B · 1.7T)
- Data: trillions tokens
- Compute: thousands GPUs

★ What is LLM vs Traditional ML?

TRADITIONAL ML	LLM
feature engineering · small models · task specific · no emergent	pretrained · massive text · general purpose · emergent · zero-shot · few-shot reasoning

TIMELINE EVOLUTION

Model	Year	Parameters
GPT-1	2018	117M
GPT-2	2019	1.5B
GPT-3	2020	175B
GPT-4	2023	1.7T estimated

Key Terms

- LLM · Large Language Model · massive text · billions parameters · predicts next token
- GPT · Generative Pre-trained Transformer
- Parameters · 7B · 13B · 70B · 175B · 1.7T · model weights
- Tokens · pieces of words
- Emergent abilities · zero-shot · few-shot reasoning
- Scale · parameters + data + compute

LLM HANDBOOK

50 Must-Know Concepts

② TRANSFORMER ARCHITECTURE - CORE OF LLMs

★ What is Transformer?

Transformer = Attention Is All You Need (2017) = Encoder + Decoder + Self-Attention = Core of LLMs

• What is Transformer Architecture?

- Input Text → Tokenization → [tokens]
- Embedding [vectors] →
- Positional Encoding → [position info]
- Transformer Blocks [N layers]
- Output [logits • next token probabilities]

• What is Transformer Block?

- Multi-Head Self-Attention [attention mechanism focuses on relevant words]
- Add & Norm [residual + LayerNorm]
- Feed Forward [MLP]
- Add & Norm [residual + LayerNorm]

Stack N times: 12 layers GPT-2,
• 96 layers GPT-3

What is Self-Attention?

Mechanism that weighs importance of each word to each other word

→ Example: "The cat sat because it was tired" → "it" attends to "cat"

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Q = Query | K = Key | V = Value

What is Multi-Head?

Multiple attention heads — each learns different relationships

- parallel heads • 12 heads • each head 64 dim

ENCODER vs DECODER

Encoder	Decoder	unidirectional
bidirectional	sees past and future	unidirectional sees only past
BERT uses encoder	GPT uses decoder —	interview trap generates next token

TRANSFORMER BLOCK

Multi-Head Self-Attention	focuses on relevant words
Add&Norm	residual + LayerNorm
Feed Forward	MLP
Add&Norm	stack N times — 12 to 96 layers

Key Terms

- Transformer = Attention Is All You Need (2017) — encoder • decoder • self-attention — core LLMs
- Input → tokenization → embedding → positional encoding → transformer blocks → output logits
- Transformer Block → Multi-Head Self-Attention • Add&Norm • Feed Forward • stack N → 12 to 96 layers
- Self-Attention → weighs importance — Q=Query • K=Key • V=Value — formula: $\text{softmax}(QK^T / \sqrt{d_k}) V$
- Multi-Head → multiple heads → different relationships → parallel → 12 heads → 64 dim
- Encoder → bidirectional → BERT vs Decoder → unidirectional → GPT → generates next token

LLM HANDBOOK

50 Must-Know Concepts

3. TOKENIZATION & EMBEDDINGS

★ What is Tokenization?

Splits text into tokens — pieces of words. Tokens are the units an LLM processes.

Types:

- Word level: hello world → tokens. But vocab large & OOV issue
- Character level: h e l l o → vocab small (256) but sequence long
- Subword BPE: Byte Pair Encoding — most common in GPT. Uses merges frequent pairs. ChatGPT uses tiktoken: hello → hel + lo. Tokens balance vocab size & sequence length.

★ What is BPE?

Byte Pair Encoding = starts with chars, merges most frequent pair iteratively. Example: hello + world → frequent merges → hel+lo etc. Vocab ~50k.

Example Tokenization

Input	Tokens	Count	
Hello world! →	Hello, world, !	3 tokens	!
ChatGPT tokenization:	Hello world →	2 tokens	—
≈ 1 token ~ 0.75 words	100 tokens ~	75 words	

★ What is Embeddings?

Converts tokens to vectors — numbers in dense representation. Meaning: token → vector [0.2, -0.5, 0.8, ...]
768 dim, 12288 dim.

How Embeddings work?

- Token ID → Embedding Matrix lookup → Vector representation. Captures semantic meaning: similar words have close vectors. Famous: king - man + woman = queen
- Types: Token Embeddings — token meaning. Positional Embeddings — position info (transformer has no recurrence, needs position: sinusoidal / learned). Segment Embeddings — for BERT.

TOKENIZATION TYPES

Type	Example	Note
Word level	hello world	large vocab, OOV
Character level	h e l l o	small vocab 256, long sequence
Subword BPE	hel + lo vocab,	balance ~50k vocab, most common — GPT tiktoken

EMBEDDING DIMENSIONS

Model	Dimension
GPT-2	768 dim
GPT-3	12288 dim
Llama	4096 dim

What is Context Window?

Max tokens a model can process at once. GPT-3: 2048 tokens • GPT-4 8k / 32k / 128k • Claude: 200k • Gemini: 1M • → Context window is crucial for long documents!

Key Terms

- Tokenization: splits text into tokens; units the model reads.
- to build vocab. *Subword BPE balances vocab size vs sequence length for efficiency. or learned.*
- Embeddings: token → vector as dense representation of meaning.
- Token ID → Embedding Matrix lookup → vector capturing semantics; king - man + woman = queen
- Token Embeddings: capture meaning. Positional Embeddings: add position info since transformers lack recurrence;
- Segment Embeddings: used in BERT for segment separation.
- Context Window: max tokens model can handle at once — GPT-3 2048, GPT-4 8k/32k/128k, Claude 200k, Gemini 1M.

LLM HANDBOOK

50 Must-Know Concepts

4

SECTION 4: PRE-TRAINING & TRAINING - HOW LLMs TRAINED

★ What is Pre-training?

Training LLM on massive text data → self-supervised predicts next token → uncovers language patterns.

★ How Pre-training works?

- Massive corpus → Common Crawl • Wikipedia + Books • GitHub → trillions tokens → Tokenization → Next Token Prediction objective: maximize $P(\text{ext token} | \text{previous tokens})$ → Gradient Descent updates weights → billions parameters → Pre-trained Base Model.

Example: Training objective

Input: "The cat sat on the _"
→ Predict: "mat" / model predicts next token
The cat sat on the mat → loss compares predicted vs actual → updates weights.

★ What is Training Data?

- Common Crawl web pages → 60%
- Wikipedia → 3%
- Books → 15%
- Code → GitHub
- ArXiv papers → reasoning
- Data quality filtering & deduplication important.

★ What is Compute?

- Pre-training needs thousands GPUs for months → 175B model ≈ 300B tokens
- 1000s A100 GPUs for weeks → cost millions
- GPT-3 ≈ \$4M • GPT-4 ≈ \$100M

★ What is Scaling Laws?

Kaplan et al → performance improves predictably with more parameters/data/compute → power law: $\text{Loss} \sim N^{-0.07}$ where N = parameters/data/compute.

Chinchilla

optimal compute → optimal model size & data size equally important → 70B model + 1.4T tokens better than 280B + 300B tokens.

PRE-TRAINING DATA

Dataset	Example	Note
Common Crawl	web	60% → huge
Wikipedia	3%	high quality
Books	15%	long context
Code	GitHub	code generation
ArXiv papers	reasoning	reasoning

★ What is Objective?

Next Token Prediction → self-supervised → maximize $P(\text{ext} | \text{previous})$ → uncovers.

Definition: model learns by predicting the next token given previous context → maximizes $P(\text{ext} | \text{previous})$

★ Key Terms

- Pre-training → massive text → self-supervised → next token prediction uncovers → learns patterns
- Corpus → Common Crawl • Wikipedia Books → GitHub → trillions tokens
- Next Token Prediction → maximize $P(\text{ext token} | \text{previous})$ objective
- Gradient Descent → updates weights
- Base Model → pre-trained foundation
- Compute → thousands GPUs → months → millions cost → GPT-3 \$4M • GPT-4 \$100M
- Scaling Laws → Kaplan → performance improves → power law $\sim N^{-0.07}$ → parameters/data/compute
- Chinchilla → optimal model size & data size equally → 70B + 1.4T tokens better than 280B + 300B

LLM HANDBOOK

50 Must-Know Concepts

⑤ FINE-TUNING & INSTRUCTION TUNING

★ What is Fine-tuning?

Definition: Takes pre-trained base model & trains further on specific task data → task-specific.

Types:

- Full Fine-tuning: updates all parameters → expensive, needs many GPUs
- Parameter Efficient Fine-tuning (PEFT): updates few parameters → efficient
 - LoRA, QLoRA

* What is Instruction Tuning?

Definition: Fine-tunes on instruction-response pairs → teaches model to follow instructions.

Example:

- Instruction: "Translate English to French: Hello" → Response: "Bonjour".
- Dataset example: Instruction: "Summarize this text..." → Response: Summary...
- Instruction: "Write code to..." → Response: code... thousands instructions.

* Why Instruction Tuning?

- Base model → completes text, does NOT follow instructions
- Instruction tuned → follows instructions, helpful, answers helpfully

Base Model vs Instruction Tuned

	Base Model	Instruction Tuned
Input:	"What is AI?"	"What is AI?"
Output:	"What is AI?" What is AI?" → repeats question, just completes	"AI is... answers helpfully" answers helpfully

* What is LoRA?

Low-Rank Adaptation (PEFT method) → freezes base model, adds low-rank matrices A & B, trainable, small params → efficient ~0.1% params

* How LoRA works?

Original weight W (frozen) → $W + BA$

- $BA = B \times A$, where $B (d \times r)$, $A (r \times k)$, $r = \text{rank (small, 8, 16)}$
- trainable params $\approx 0.1\%$ → memory efficient ~ fine-tune 7B model on single GPU

FULL vs LoRA vs QLoRA			
Full Fine-tuning		LoRA	QLoRA
Params	all params updated	low-rank 0.1% params, base frozen + BA	quantized LoRA 4-bit
Cost	expensive, many GPUs, 100% params	small, memory efficient, single GPU	even more efficient, 7B on consumer GPU

LoRA → recommended

* What is QLoRA?

Quantized LoRA → 4-bit quantization → even more memory efficient → fine-tune 65B on single 48GB GPU

Key Terms

- Fine-tuning: pre-trained base → further trained on task-specific
- Full Fine-tuning: all params → expensive, many GPUs, 100%
- PEFT: Parameter Efficient → few params, efficient → LoRA, QLoRA
- Base: completes/repeats vs Instruction Tuned: answers helpfully, follows |
- LoRA: Low-Rank Adaptation → freezes base, adds low-rank BA, trainable ~0.1%, $W+BA$, $B (d \times r)$, $A (r \times k)$, $A (r \times k)$, rank 8/16
- QLoRA: quantized → 4-bit → even more efficient → 65B on single 48GB GPU

LLM HANDBOOK

50 Must-Know Concepts

6 RLHF & ALIGNMENT - MAKING LLMs HELPFUL SAFE

★ What is RLHF?

Reinforcement Learning from Human Feedback aligns LLM with human preferences → helpful • harmless • honest.

★ What is? Why RLHF?

Pre-trained + instruction-tuned still may generate harmful • biased • not aligned with human values. RLHF aligns model with human preferences for safety & helpfulness.

RLHF 3 Steps:

1. Step 1 — Supervised Fine-Tuning (SFT): instruction tuning on human-written responses.
base model → SFT model →
2. Step 2 — Reward Model Training (RM): train model to predict human preferences. humans rank responses best→worst. RM learns to score. RM = human preference model.
3. Step 3 — Reinforcement Learning (RL) with PPO: Proximal Policy Optimization. SFT model generates responses → RM scores → PPO updates SFT to maximize RM score → aligned model.

RLHF = SFT + Reward Model + RL
PPO = Aligned • Helpful • Safe

Example: Human Preference

Question: Explain AI safety.

Response A: AI is helpful, safe, factual, avoids harm.

Response B: AI is harmful, biased, misleading info.

Human ranks: A > B. RM learns A is better → RM score(A) > score(B).

RL/PPO optimizes to produce A-like responses.

★ What is PPO?

Proximal Policy Optimization — RL algorithm. Updates policy/model to maximize reward/RM score while staying close to original SFT policy. Prevents collapse/large changes via KL penalty.

★ What is Alignment?

Making LLM helpful • harmless • honest — the 3

- Helpful: follows instructions, useful, accurate
- Harmless: not harmful, not biased, not illegal, safety priority.
- Honest: truthful, factual, avoids hallucinations.

★ What is DPO?

Direct Preference Optimization — alternative to RLHF. Simpler, no reward model. Directly optimizes on preference data. DPO loss = $-\log \text{sigmoid}(\text{RM}_{\text{preferred}} - \text{RM}_{\text{rejected}})$.
More stable than PPO, modern preferred.

RLHF vs DPO

RLHF	DPO
3 steps: SFT + RM + RL PPO • complex • needs RM • PPO + KL penalty	direct preference • no RM • simpler • stable • no PPO • directly optimizes preference • more stable • modern preferred

Key Terms

- RLHF: Reinforcement Learning Human Feedback — aligns to be helpful • harmless • honest via human preferences
- Why: pre-trained+instruction still harmful/biased/not aligned with humans.
- 3 Steps: SFT = instruction tuning on human-written → SFT; RM = predicts human prefs via human ranking best→worst → scores; RL+PPO = SFT generates → RM scores → PPO updates to maximize RM → aligned.
- PPO: RL algo maximize RM reward, stay close to SFT via KL penalty prevents collapse. - (3H): helpful = no ham/bias/illegal; honest = truthful/no hallucinate.
- DPO: Direct Preference Optimization — alternative, no RM, directly $-\log \text{sigmoid}(\text{RM}_{\text{pref}} - \text{RM}_{\text{reject}})$ → more stable • modern preferred.
- Constitutional AI: Anthropic method — model follows written constitution/principles, self-critique & revise to be safe.

LLM HANDBOOK

50 Must-Know Concepts

7. PROMPT ENGINEERING - MOST IMPORTANT FOR USERS

★ What is Prompt Engineering?

Crafting inputs to get best outputs from LLMs — art and science.

Prompt Engineering = Crafting Inputs + Best Outputs — Art + Science + Most Important for Users

PROMPTING TECHNIQUES

Technique	Description	Example
Zero-Shot	no examples, just instruction	Translate: Hello → Bonjour
Few-Shot	few examples, learns examples	English → (few examples)
CoT	Let's think step by step improves — reasoning most important	— step-by-step improves reasoning
Role	You are expert —	You are a tutor: explain
Instruction	clear & specific —	Summarize 3 bullets
Format	controls output format	Output: JSON / table

★ What is System vs User Prompt?

System prompt: sets behavior, hidden role/instructions vs User prompt: the user's actual query, e.g. "What is AI?"

TEMPERATURE

Temp	Meaning	Effect
0	deterministic, low	consistent & predictable
0.7	balanced —	recommended
1.0	creative, random, high	more varied & random

★ What is Top-p & Top-k?

Top-p (nucleus sampling): top-p = top-p probability mass, e.g. 0.9 → 90% probability mass

Top-k: top k tokens selected, e.g. k = 50 → top 50 tokens for diversity

Key Terms

- Prompt Engineering: crafting inputs → best outputs, art + science, most important — Translate: Hello → Bonjour
- Chain-of-Thought (CoT): Let's think step by step — step-by-step reasoning, improves reasoning, most important
- Role: You are expert — sets role/context • Instruction: clear & specific — Summarize 3 bullets
- Format: Output JSON / table — controls output format
- System prompt: sets behavior, hidden role vs User prompt: user query — What is AI?
- Temperature: controls randomness — 0 deterministic/low, 0.7 balanced (recommended), 1.0 creative/high
- Top-p: nucleus sampling — top-p probability mass, 0.9 = 90%
- Top-k: top k tokens — 50 → top 50 for diversity

LLM HANDBOOK

50 Must-Know Concepts

SECTION 8 • RAG - RETRIEVAL AUGMENTED GENERATION

8

★ What is RAG?

Combines retrieval search + generation. LLM retrieves relevant docs then generates answer using docs.
Reduces hallucination.
Uses external knowledge instead of only training data.

RAG = Retrieval Search + Generation
LLM + External Knowledge + Reduces Hallucination +
★ Most Important

★ Why RAG?

- LLM knowledge cutoff → outdated info
 - Hallucinates → makes up facts
 - No private data knowledge
- RAG solves: retrieves fresh/private data, reduces hallucination, cites sources.

WITHOUT RAG vs WITH RAG

Without RAG	With RAG
• Cutoff outdated	• Fresh private data
• Hallucinates / made-up	• Reduces hallucination
• No private data	• Cites sources
• No sources	• Up-to-date — ★ must know

★ Example:

Without RAG

Q: What is our company revenue 2023?
LLM: I don't know. Cutoff 2021 → hallucinate/fake.

With RAG

Retrieves doc → revenue 2023 **\$10M**
LLM: Revenue 2023 is \$10M based on retrieved doc, cites [source.pdf].

★ How RAG Works? — Flow:

Documents (PDFs / DB) →

Chunking → split into chunks → overlapping 50 tokens 50 tokens →

Embeddings → embed chunks to vectors →

Vector DB → store vectors (Chroma • Pinecone • Weaviate • FAISS)

Query → user query is embedded to query vector →

Retrieval → similarity search, top-k chunks relevant →

Augment → prompt + retrieved chunks as context + query →

Generation → LLM generates answer using context + query, cites sources.

★ RAG Components

Chunking: split docs → 500 tokens, overlap 50 tokens

Embeddings: model → text-embedding-ada-002 • sentence-transformers

Vector DB: Chroma • Pinecone • Weaviate • FAISS — cosine similarity

Retriever: top-k similarity search → k=3 or 5

Generator: LLM → GPT-4 • Llama — generates using context

★ What is Advanced RAG?

- Query rewriting → improves query for better retrieval
- HyDE → hypothetical document embeddings
- Re-ranking → re-ranks chunks for better relevance
- Parent Document Retriever → small chunks retrieve but return parent large chunk
- Self-RAG → LLM decides when to retrieve

★ KEY TERMS — Purple box

RAG = Retrieval Augmented Generation = retrieval search + generation LLM + external knowledge + reduces hallucination + most important. Why: cutoff outdated, hallucinates, no private data, no sources → solves: fresh/private, cites sources, reduces hallucination. Flow: Documents → Chunking 500t overlap 50 → Embeddings vectors → Vector DB (Chroma/Pinecone/Weaviate/FAISS) → Query embed → Retrieval top-k → Augment prompt context+query+retrieved chunks → With transformers, Vector DB Chroma/Pinecone/Weaviate/FAISS cosine, Retriever top-k k=3/5, GPT-4 Advanced: Query keyword rewriting, HyDE hypothetical, Re-ranking, Parent Doc small→parent large, Self-RAG decides when retrieve.

LLM HANDBOOK

50 Must-Know Concepts

SECTION 9: VECTOR DBs & EMBEDDINGS

★ What is Embeddings?

Converts text to vectors numbers • captures meaning → text → vector
[0.2, -0.5, 0.7, ...] • 768 dim •
• similar meaning → close vectors

Embedding Models:

- text-embedding-ada-002 • OpenAI • 1536 dim • popular
- all-MiniLM-L6-v2 • sentence-transformers • 384 dim • free • small
- all-mpnet-base-v2 • 768 dim • free • good quality

★ What is Vector DB?

Database stores vectors • efficient similarity search.

★ How Similarity Search?

Query vector compared to stored vectors • cosine similarity • dot product • Euclidean distance → finds closest.

★ Cosine Similarity

$\cos = \frac{A \cdot B}{|A||B|}$ • range -1 to 1 • 1 similar • 0 not similar • -1 opposite • most common for text

VECTOR DBs comparisinn

Name	Type	Notes
Chroma	open source chroma.Client()	local easy Python popular for RAG
Pinecone	managed cloud	scalable fast
Weaviate	open source	GraphQL
FAISS	Facebook AI	Sim. Search library
	not DB, fast	local -1
Qdrant	open source	Rust fast
Milvus	open source	large scale

★ What is Chunking?

Splitting docs for embeddings.

Strategies:

- Fixed Size: 500 tokens overlapping 50 • simple
- Recursive: splits by paragraphs/sentences
- respects structure better • embedding similar

★ What is Best Chunking?

500-1000 tokens overlapping 50-100 • recursive recursive preferred

★ What is Indexing?

- HNSW = Hierarchical Navigable Small World • fast approximate nearest neighbor ANN search • not exact but fast

Key Terms:

Embeddings: text → vectors numbers meaning • similar → close vectors • king-man+woman=queen

Models: ada-002 OpenAI 1536d popular • all-MiniLM 384d free small • mpnet 768d free good

Vector DB: stores vectors efficient similarity search

Similarity: cosine/dot/Eucl. closest • cosine $\cos = \frac{A \cdot B}{|A||B|}$ -1 → 1 • 1 similar • most common • < common for text

DBs: Chroma open source local easy Python popular RAG rec. • Pinecone managed cloud scalable fast • Weaviate GraphQL • FAISS lib Facebook fast local • Qdrant Rust fast • Milvus large scale

Chunking: splitting docs → embeddings • Fixed: 500 ov 50 simple • Recursive: para/sentences respects better • Semantic: meaning emb sim • Best: 500-1000 ov 50-100 recursive preferred

Indexing: HNSW fast ANN approx not exact fast

LLM HANDBOOK

50 Must-Know Concepts

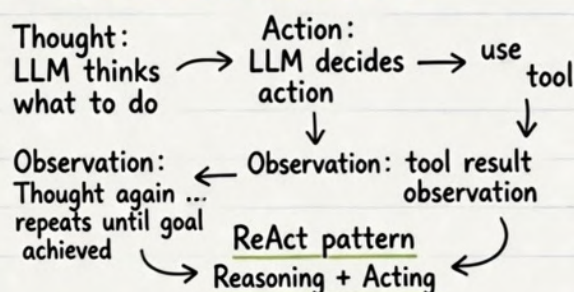
10. LLM AGENTS

★ What is LLM Agent?

Agent = LLM + Reasoning + Planning + Tools + Actions + Autonomous Goal

LLM that can reason, plan, use tools, take actions autonomously to achieve goal, not just generate text.

★ How Agent Works?



Example Agent Goal: Book flight to Paris

Steps	
- Thought:	Need to search flights
- Action:	search_flights(Paris)
- Observation:	Flights found \$500
- Thought:	Need to book
- Action:	book_flight(flight_id)
- Observation:	Booked, Goal achieved.

Key Terms

- Agent = LLM + reasoning + planning + tools + actions + autonomous goal — not just text
- Loop: Thought thinks → Action decides tool → Observation result → Thought again → repeats — ReAct Reasoning+Acting
- Example Goal: Book flight — Thought: search → Action: search_flights → Observation: flights → Thought → Observation: Booked → Goal achieved
- Components: LLM brain GPT-4 • Tools functions: search/calculator/long-term vector DB • Planning: breaks goal • chain-of-thought (tree-of-thought)
- Tools: Search search(query) • Calculator expression • Code Execute code • API: weather, flight • useAI: multi-agent creation • CrewAI: multi-agent
- LLM: generates text • passive • no tools — vs Agent: reasons • plans • uses tools • actions • autonomous loop
- Function Calling: LLM calls functions • OpenAI defines functions • decides when to call with search_flights(destination: Paris))

★ Components

- LLM brain: GPT-4 reasoning
- Tools: functions agent can use: search • calculator • code • API • flight booking
- Memory: short-term conversation • long-term vector DB
- Planning: breaks goal into steps • chain-of-thought • tree-of-thought

★ Tools Examples

- Search: search(query)
- Calculator: calculator(expression)
- Code Execute: execute_code(code)
- API calls: weather API • flight API
- Database: query DB

★ Frameworks

- LangChain: most popular • agents • tools • chains
- LangGraph: graph-based agents • stateful
- AutoGPT: autonomous agent • goal-driven
- BabyAGI: task creation • prioritization
- CrewAI: multi-agent collaboration

LLM vs	AGENT
- generates text - predicts next token - passive - no tools	- reasons • plans - uses tools - takes actions - autonomous • goal-driven loop - Thought → Action → Observation
← must know →	

★ What is ReAct?

Reasoning + Acting pattern: Thought → Action → Observation loop

★ What is Function Calling?

- LLM can call functions • OpenAI function calling • defines functions
- * functions=[{name: search_flights, description: "Search flights", parameters: {destination: string}}]
- * LLM decides to call search_flights({destination: Paris}).
- ★ ReAct: decides to call passive • Thought — vs Agent: reasons • plans • BabyAGI: task creation • flight → Observation — must know

LLM HANDBOOK

50 Must-Know Concepts

11 SECTION 11 EVALUATION & BENCHMARKS

★ What is Metrics?

- **Perplexity**: how well predicts next token — lower better
- **Accuracy**: correct answers / total
- **F1**: precision and recall balance
- **BLEU/ROUGE**: for translation summarization — n-gram overlap
- **Human Evaluation**: human rates quality helpfulness — most reliable but expensive

• What is Good Score?

- MMLU: GPT-3 43% → GPT-4 86% improvement
- GSM8K: GPT-3 15% → GPT-4 92% huge improvement in reasoning

BENCHMARKS

Benchmark	Focus	Notes/Signal
MMLU	57 subjects knowledge	0-100% — GPT-4 86%
GSM8K	8K math	math reasoning
HumanEval	code	164 coding
HellaSwag	commonsense	reasoning
TruthfulQA	truthfulness	hallucination
BIG-Bench	diverse	200+ tasks
Chatbot Arena	human preference	ELO — human votes

• What is Contamination?

Benchmark data in training data → model memorized not truly reasoning → leaked data ⇒ inflated scores. →

• What is Hallucination?

LLM generates false information — confident but wrong.
Types: Intrinsic — contradicts source vs Extrinsic — cannot verify from source

• How to Reduce Hallucination?

- RAG: retrieves docs → reduces hallucination
- Lower temperature: less random → more consistent
- Prompting: instruct — don't hallucinate, cite sources
- Fine-tuning: on truthful data
- Fact checking: verify outputs

Key Terms — Evaluation metrics, benchmarks measure performance. Metrics: Perplexity lower better predicts next token; Accuracy correct/total; F1 precision recall; BLEU/ROUGE translation summarization n-gram; Human Evaluation human rates helpfulness reliable expensive. Benchmarks: / subjects knowledge 0-100% GPT-4 86%; GSM8K math 8K reasoning; HumanEval code 164 coding; HellaSwag commonsense; TruthfulQA truthfulness hallucination; BIG-Bench diverse 200+ tasks; Chatbot Arena human preference ELO votes — must know. Good Score: MMLU GPT-3 43% → GPT-4 86%; GSM8K GPT-3 15% → GPT-4 92% improvement. Contamination: benchmark in training → memorized leaked → inflated scores. Hallucination: false information confident wrong — Intrinsic contradicts · Extrinsic cannot verify. Reduce: RAG retrieves reduces; Lower temperature less random; Prompting don't hallucinate cite; Fine-tuning truthful; Fact checking.

LLM HANDBOOK

50 Must-Know Concepts

12 LLM OPTIMIZATION & QUANTIZATION

★ What is Optimization?

Makes LLMs faster, smaller, cheaper to run.

★ Why Optimization?

LLMs large 7B, 70B parameters → needs many GPUS, memory expensive & slow. Optimization reduces memory, speeds up inference, lowers costs.

Techniques:

Quantization: reduces precision of weights from 16-bit → 4-bit/8-bit → memory 4x smaller & faster

Pruning: removes unimportant weights/connections → 50% weights removed → smaller & faster

Distillation: trains small student model to mimic large teacher model → small model learns from large teacher → 7B student mimics 70B teacher → smaller faster

★ KV Cache:

Caching Key-Value cache for attention speeds up generation, avoids recomputing past tokens → cache past K & V

★ Flash Attention:

optimized attention implementation → faster, memory efficient, IO-aware tiling

★ Speculative Decoding:

small draft model generates drafts, large model verifies in parallel → faster 2x speed

★ What is Quantization Types?

FP32 32-bit float → full precision, large | FP16 16-bit half → 2x smaller, faster, mixed precision training | INT8 8-bit integer → 4x smaller, slight loss | INT4 4-bit → 8x smaller, more loss, QLoRA uses 4-bit

QUANTIZATION

Type	Details
FP32	32-bit, full precision, large
FP16	16-bit half, 2x smaller, faster, mixed precision
INT8	8-bit integer, 4x smaller, slight loss
INT4	4-bit, 8x smaller, more loss, QLoRA 4-bit

★ What is GPTQ?

Quantization method → accurate 4-bit quantization.

★ What is AWQ?

Activation-aware Weight Quantization → better than GPTQ, preserves important weights.

★ What is vLLM?

Fast inference engine → PagedAttention, continuous batching → faster serving.

Key Terms:

- Optimization → faster, smaller, cheaper • Why → large 7B/70B, many GPUS, memory expensive
- Quantization → reduces precision 16-bit → 4-bit/8-bit, 4x smaller & faster
- Pruning → removes unimportant, 50% weights removed
- Distillation → student mimics teacher, 7B mimics 70B
- KV Cache → Key-Value cache speeds generation, avoids recompute
- Flash Attention → optimized, IO-aware tiling • Speculative Decoding → draft+verify parallel, faster 2x • FP32 32-bit full vs FP16 16-bit half vs 8-bit 4x vs INT4 4-bit 8x QLoRA efficient → accurate 4-bit
- AWQ → activation-aware, better preserves
- vLLM → PagedAttention, continuous batching

LLM HANDBOOK

50 Must-Know Concepts

13 LLM SECURITY & SAFETY

★ What is LLM Security?

Protecting LLMs from attacks and ensuring safe outputs.

Attacks

- ★ Prompt Injection — attacker injects malicious prompt "Ignore previous instructions..." hijacks LLM
- ★ Jailbreaking — bypass safety measures DAN Do Anything Now prompt makes LLM bypass safety generates harmful
- Data Poisoning — poison training data malicious data makes model harmful harmful biased
- Model Inversion — extracts training data privacy leak
- Adversarial Attacks — small perturbations makes model wrong

Defenses

- ★ Input Validation — sanitizes prompt checks malicious
- Output Filtering — filters harmful outputs

★ What is Safety?

Ensuring LLM outputs safe helpful honest not harmful.

Safety Techniques:

- ★ RLHF — aligns helpful harmless honest human preferences
- Constitutional AI — follows constitution principles self-critique Anthropic
- Content Filtering — filters harmful illegal biased output
- Red Teaming — tests LLM for vulnerabilities adversarial testing team tries to break
- Guardrails — NeMo Guardrails Llama Guard checks inputs outputs safe

★ What is Privacy?

Protecting training data and user data.

Privacy Techniques:

- ★ Differential Privacy — adds noise to training data privacy
- Federated Learning — trains on decentralized data no central data
- ★ Data Anonymization — removes PII personally identifiable identifiable information
- No training on user data — don't train on private user data

ATTACKS vs DEFENSES

Attack	Example	Defense
Prompt Injection	Ignore previous hijacks	Input Validation sanitizes checks malicious
Jailbreaking	DAN bypass safety	Output Filtering filters harmful
Data Poisoning	poison training data harmful	Data validation filtering
Model Inversion	extracts training data privacy leak	Differential Privacy adds noise

What is Responsible AI? Ethical AI development fairness transparency accountability privacy safety

Purple Key Terms box

Security — protecting from attacks safe outputs; Attacks — Prompt Injection: Ignore previous hijacks LLM; Jailbreaking: DAN Do Anything Now bypass safety harmful; Data Poisoning: poison training harmful biased; Model Inversion: extracts training data privacy; Adversarial: small perturbations wrong; Defenses — Input Validation: sanitizes checks malicious; Output Filtering: filter limiting: limits requests; Sandboxing: isolates safe; safe helpful honest not harmful; RLHF: helpful harmless honest; Constitutional AI: Content Filtering: filters harmful illegal biased; Red Teaming: training user data; Differential Privacy: federating: decentralized no central; Anonymization: removes PII; No Responsible — ethical fairness transparency accountability privacy safety.

LLM HANDBOOK

50 Must-Know Concepts

14 LLM FRAMEWORKS & TOOLS

BUILDING
LLM APPS

★ What is? What are Frameworks?

Tools to build LLM applications easier.

Framework	Description
• LangChain apps	most popular framework for LLM chains agents RAG tools memory Python
• pip install	langchain
• LlamaIndex	data framework for RAG indexing retrieval connects data to LLM formerly GPT-Index
• LangGraph	graph-based agents stateful workflows built on LangChain
• Semantic Kernel	Microsoft framework for LLM apps C# Python
• Haystack	open source LLM framework pipelines
• Guidance	Microsoft controls LLM outputs structure

Tool	Description
• OpenAI API	GPT-4 API most powerful API gpt-4o gpt-4-turbo
• Hugging Face	Transformers library for open source models Llama Mistral
• Ollama	runs LLMs locally easy ollama run
• LM Studio	runs LLMs locally GUI easy
• vLLM	fast inference PagedAttention fast serving
• Text Generation Inference TGI	Hugging Face inference server

Vector DBs | Chroma • Pinecone • Weaviate • FAISS • Qdrant

- Chroma: local easy Python popular RAG,
- Pinecone: managed cloud scalable,
- Weaviate: open source GraphQL,
- FAISS: library Facebook fast local,
- Qdrant: Rust fast

What is LangChain Components?

Component	Description
• Models	LLM wrapper ChatOpenAI
• Prompts	PromptTemplate
• Chains	combines components LLMChain
• Agents	uses tools ReAct
• Memory	remembers conversation ConversationBufferMemory
• Retrievers	retrieves docs VectorStoreRetriever
• Tools	functions search calculator

Example LangChain RAG

```
Code:
from langchain.document_loaders import TextLoader
loader = TextLoader("data.txt")
docs = loader.load()
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=500)
chunks = splitter.split_documents(docs)
from langchain.embeddings import OpenAIEmbeddings
embeddings = OpenAIEmbeddings()
from langchain.vectorstores import Chroma
db = Chroma.from_documents(chunks, embeddings)
retriever = db.as_retriever()
from langchain.chains import RetrievalQA
chain = RetrievalQA.from_chain_type(llm=ChatOpenAI(),
retriever=retriever)
chain.run("What is revenue?") # RAG chain
```

Key Terms

- Frameworks: tools build LLM apps easier • LangChain: most popular chains agents RAG tools memory pip install • LlamaIndex: data framework RAG indexing retrieval connects data to LLM formerly GPT-Index
- LangGraph: graph-based agent stateful workflows built on LangChain
- Semantic Kernel: Microsoft framework C# Python • Haystack:
- Guidance: Microsoft controls LLM outputs structure • Tools: OpenAI API-4o
- Tools: gpt-4o voice to text open source • library LlamaX Transformers pipeline
- Hugging Face LLMs local pipeline • Ollama: runs LLMs locally easy
- LM Studio: runs LLMs locally GUI easy edit run the doc first fast local,
- Vectorstore cloud scalable • FAISS: library Facebook fast local
- LangChain Components: Models ChatOpenAI, Prompts PromptTemplate,
- Example RAG: TextLoader load, RecursiveCharacterTextSplitter chunk_size=500, OpenAIEmbeddings, a Chroma.from_documents, Chroma.from,
- docs, retriever as_retriever, RetrievalQA from_chain_type, chain, chain run "What is revenue?" RAG chain

LLM HANDBOOK

50 Must-Know Concepts

15 LLM PROJECTS PORTFOLIO

★ What is?

Projects to build portfolio get LLM Engineer job

15. LLM PROJECTS PORTFOLIO - MUST BUILD

- ★ **Project 1: RAG Chatbot with Your Data:**
PDFs → Chunking → Embeddings → Vector DB Chroma RetrievalQA chain. Chat with PDFs Streamlit UI.
wow factor very high best for portfolio must have.
- ⚙️ **Project 2: AI Agent with Tools:**
LangChain agent with tools search calculator code execution ReAct loop. Goal based autonomous agent. Tools: search web calculator code AutoGPT style.
wow very high.
- 🧠 **Project 3: Fine-tune LLM with LoRA:**
Take Llama 7B fine-tune on custom dataset with LoRA QLoRA on single GPU custom chatbot domain specific
wow high shows fine-tuning skills.

⚡ **Project 4: LLM + Function Calling:**
OpenAI function calling LLM decides when to call function flight booking weather API example functions search_flights book_flight LLM calls based on query wow high modern skill.

🚀 **Project 5: AI SaaS with LLM:**
Build SaaS using LLM example AI content generator AI code reviewer AI resume checker Next.js + OpenAI API + Stripe payment
wow very high can monetize startup potential.

📺 **Project 6: Chat with YouTube Video:**
YouTube transcript → Chunking → Embeddings → Vector DB → RAG chain Ask questions about video content wow high popular.

PROJECT	SKILLS	WOW FACTOR
RAG Chatbot	PDFs Chunking Embeddings Vector DB Chroma	very high must have best portfolio
AI Agent	Tools search calculator code ReAct LangChain	very high autonomous
Fine-tune LoRA	Llama 7B custom dataset QLoRA single GPU	high fine-tuning skills
Function Calling	OpenAI functions search_flights book_flight LLM decides	high modern
AI SaaS	Next.js OpenAI API Stripe monetize	very high startup potential
Chat YouTube	transcript Chunking Embeddings Vector DB RAG Ask video	high popular

Portfolio Tips:

- GitHub with code README demo video
- Streamlit / Hugging Face Spaces deployed live demo
- LinkedIn posts explain projects
- Clean repo: requirements.txt, docs, screenshots
- Add example prompts & sample data
- Record 1-2min demo video for each

Key Terms

• RAG Chatbot • PDFs • Chunking • Embeddings • Vector DB • Chroma • RetrievalQA • Streamlit • very high • must have best portfolio; AI Agent • Tools • search • calculator • code • ReAct • LangChain • AutoGPT • very-tune LoRA • Llama 7B • custom dataset • QLoRA • single GPU • domain specific • high • fine-tuning; Function Calling • OpenAI functions • flight books • flight • modern; AI SaaS • Next.js • OpenAI API • Stripe • monetize • high • startup potential; Chat YouTube • transcript • Chunking • Embeddings • Vector DB • RAG • Ask video • high • popular; Portfolio GitHub • README • demo video • Streamlit • Hugging Face Spaces • deployed • LinkedIn posts • explain.

LLM HANDBOOK

16/20

50 Must-Know Concepts

16

SECTION NAME: LLM INTERVIEW QUESTIONS - MUST KNOW TRAPS

★ What are Top LLM Interview Questions?

Q1. What is Transformer?

Attention Is All You Need — 2017 — encoder-decoder, self-attention. Q, K, V. $\text{Attention}(Q, K, V) = \text{softmax}(QK^T \sqrt{d_k}) V$. Multi-Head core of LLMs.

Q2. What is Self-Attention?

Weighs importance of each word to each other. Example: "The cat sat because it was tired" → it attends to "cat". Q = Query, K = Key, V = Value.

Q3. What is RAG?

Retrieval Augmented Generation = retrieval/search + generation using LLM with external knowledge to reduce hallucination. Flow: Docs → Embeddings → Vector DB → Retrieval → Augmentation → Generation + cites sources. **must know favorite trap**

Q4. What is Fine-tuning vs RAG?

Fine-tuning: updates model weights, task-specific, needs training data, expensive. vs RAG: retrieves external knowledge, no training, cheaper, fresh data, private data. Use RAG when data changes often / private data → cheaper, Use Fine-tuning when need style/behavior change, deep task-specific.

Q5. What is LoRA / QLoRA?

LoRA = Low-Rank Adaptation (PEFT). Freezes base model, adds low-rank BA, ~0.1% params, efficient, single GPU. QLoRA = quantized 4-bit, even more efficient, 65B on single GPU.

Q6. What is RLHF? ★

Reinforcement Learning from Human Feedback. 3 steps: 1) SFT = instruction tuning. 2) Reward Model = human preferences ranking. 3) RL (PPO) = maximizes RM score → aligns helpful / harmless / honest. DPO = alternative, simpler, no RM, direct preference.

Q7. What is Prompt Engineering?

Crafting inputs for best outputs. Zero-Shot = no examples. Few-Shot = few examples. CoT = "Let's think step by step" → step-by-step reasoning improves. Role = "You are an expert". Temperature: 0 = deterministic, 0.7 = balanced (recommended), 1.0 = creative.

Q8. What is Hallucination?

False information presented as fact, confidently wrong. Intrinsic = contradicts source context. Extrinsic = cannot be verified. Reduce by: using RAG + retrieve sources, lower temperature, prompt "don't hallucinate", cite sources, fine-tuning for factual accuracy, fact-checking.

Q9. What is Quantization?

Reduces precision from 32-bit → 4/8-bit to save memory → 4x smaller, faster, slight accuracy loss. FP32 = 32-bit full. FP16 = 16-bit half → 2x. INT8 = 8-bit → 4x smaller, slight loss. Frameworks: LangChain, LangGraph, Llama 4 = 4-bit Q, AWQ, where former freeze + bias too toxic, post-training: search, calculator, code, API. Llama 4 = LangGraph, AutoGPT. vs. LLM alone = generates text, passive for fast inference.

Q10. What is LLM Agent?

QUESTION	KEYWORDS	IMPORTANCE
Transformer	Attention Is All You Need, encoder-decoder, self-attention, Q/K/V, softmax	must know
Self-Attention	weighs importance, Q/K/V connections	core mechanism
RAG	Retrieval Augmented Generation, retrieval + generation, Doc → Chunking → Embeddings → VectorDB → Retrieve → Augment → Generate	must know / favorite trap
Fine-tuning vs RAG	updates weights vs retrieves external, no training, fresh/private data	cheaper / task-specific
LoRA / QLoRA	Low-Rank 0.1%, frozen base + BA, 4-bit	efficient
RLHF	SFT + RM + RL (PPO) aligns helpful/harmless/honest, DPO simpler no RM	alignment

Key Terms

Transformer: Attention Is All You Need (2017), encoder-decoder, self-attention, Q/K/V, softmax, Multi-Head is core. • Self-Attention: weighs importance of words: example "The cat... it attends to cat": Q=Query, K=Key, V=Value. RAG: Retrieval Augmented Generation — retrieval + generation using external knowledge to reduce hallucination: flow: Docs → Chunkings → Vector DB → Retrieval → Augmentation → Generation. • Fine-tuning: updates model weights for task-specific behavior, requires training data, expensive vs RAG: retrieves external knowledge, no training, cheaper, ~~vs~~ use tuning for style/behavior/adaptation. • LoRA: Low-Rank Adaptation: freeze base, add low-rank BA, efficient for single GPU. QLoRA: quantized version, enables 65B model on single setec via RAG, lower temperature, instruct "no hallucination", cite sources, fine-tune for truthfulness. • Quantization: reduce model precision 32-bit → 4/8-bit to lower memory and speed up inference, 4x smaller faster with slight loss: FP32/FP16/INT8/INT4. • Agent: LLM with reasoning + planning + tool use + actions for autonomous loop: Thought → Action → Observation. Frameworks vs LLM = text generation only, passive.

LLM HANDBOOK

17/20

50 Must-Know Concepts

17

SECTION NAME: LLM SCALING & INFRA - DEPLOYING LLMs

★ What is LLM Scaling?

- Serving LLMs for millions users low latency high throughput cost efficient.
- Challenges: Large models 7B • 70B memory heavy needs GPUs slow inference expensive.

How to Scale? Techniques:

- Quantization 16-bit to 4-bit/8-bit memory 4x smaller faster INT4 QLoRA
- vLLM fast inference engine PagedAttention continuous batching 24x faster than HF
- Text Generation Inference TGI Hugging Face inference server optimized
- TensorRT-LLM NVIDIA optimized inference fast
- Model Parallelism split model across multiple GPUs tensor parallelism pipeline parallelism 70B needs 2 GPUs 175B needs 8 GPUs

★ What is Inference Optimization?

- KV Cache caches Key Value past tokens avoids recomputing speeds up generation 2x faster
- Continuous Batching batches requests dynamically not
- Flash Attention optimizing for batch to complete vLLM darts
- Speculative Decoding small draft model generates draft large verifies parallel

★ What is Deployment?

- Cloud AWS SageMaker GCP Vertex AI Azure ML managed LLM hosting
- Self-hosted own GPUs Kubernetes EKS with vLLM
- Edge deployment small models on phone 7B quantized 4-bit runs on phone

★ What is Cost?

GPT-3 175B inference 1k tokens ~ \$0.02
OpenAI pricing vs Self-hosted 7B GPU cost \$0.50 per hour A100

MODEL SIZE	GPUs	COST
7B	needs 1x A100 16GB	\$0.50/hr
13B	needs 1x A100 40GB open source recommended	\$1.50/hr
70B	needs 2x A100 80GB	\$4/hr
175B	needs 8x A100 80GB	\$16/hr

INFERENCE ENGINES

ENGINE	DETAILS
vLLM	fast PagedAttention continuous batching 24x faster open source recommended
TGI	Hugging Face inference server optimized
TensorRT-LLM	NVIDIA optimized fast
Hugging Face Transformers	baseline slow

★ Key Terms

Scaling = serving millions low latency high throughput cost efficient • Challenges large models 7B 70B memory heavy needs GPUs
• vLLM fast PagedAttention continuous batching 24x faster recommended → TGI Hugging Face inference server optimized • TensorRT-LLM
• NVIDIA optimized fast Model Parallelism split across multiple GPUs tensor + pipeline 2 GPUs • KV Cache caches Key Value past tokens
• Continuous Batching dynamic batches not waiting vLLM • Flash Attention optimized IO aware faster memory efficient
• Speculative Decoding small draft generates draft large verifies parallel 2x faster • Deployment Cloud Cloud Managed AWS SageMaker Cloud Azure
• managed vs Self-hosted own GPUs Kubernetes EKS vLLM vs Edge small quantized 4-bit phone • vLLM mid LLM quantized 4-bit phone
• GPT-3 175B 1k tokens \$0.02 vs Self-hosted 7B \$0.02 A100 16GB \$0.50/hr 1B \$0.50/hr
• Model Size vs 13B 1x A100 40GB \$1.50/hr 175B 8x A100 80GB \$16/hr

50 Must-Know Concepts

SECTION NAME: LLM TRENDS & FUTURE - WHAT'S NEXT

★ What is Future?

- AGI: Artificial General Intelligence → human-level AI → 2027-2030 predicted
- Superintelligence: beyond human capabilities
- World Models: LLMs understand world physics / causality
- Embodied AI: robots with LLMs for real-world tasks

★ What is Challenges?

- **Hallucination**: false info / incorrect outputs
- **Bias**: unfair / biased responses
- **Privacy**: data leakage risk
- **Compute cost**: millions to train / expensive compute
- **Alignment**: helpful / harmless / honest → hard to achieve

CLOSED vs OPEN SOURCE

Closed:	GPT-4 / Claude / Gemini → most powerful → closed → API → expensive → API cost
Open Source:	Llama / Mistral / Falcon → open weights → customizable → free → community → fine-tunable → local → privacy

note: open source recommended for learning

TRENDS

Multimodal	→ text+image+audio+video	→ GPT-4o / Gemini 1.5
Long Context	→ 1M	→ Gemini 1.5 → 10M coming → codebase/book
Small SLMs	→ Phi-3 3.8B	→ phone → efficient
MoE	→ Mixtral 8x7B	→ 8 experts / 2 active → efficient → 47B / 13B active
Reasoning	→ o1	→ chain-of-thought → inference → thinks before → better reasoning
Agents	autonomous	→ tools → planning → AutoGPT / CrewAI

Key Terms

- Trends 2024-2026: Multimodal text+image+audio+video → GPT-4o / Gemini 1.5, Long Context 1M → Gemini 1.5 → 10M coming
- codebase/book, Small SLMs → Phi-3 3.8B → phone efficient, MoE Mixture of Experts → few active per token → Mixtral 8x7B → 8 experts / 2 active
- efficient → 47B / 13B active Reasoning o1 → chain-of-thought → inference → thinks before → better reasoning. Agents → autonomous tools → planning →
- AutoGPT / BabyAGI / CrewAI → Future: AGI-raw → 2027-2030 → Superintelligence beyond human → World Models → physics →
- Embodied → robots + LLMs, Challenges: Hallucination false → Bias unfair → Privacy leakage → Compute cost millions → Alignment
- helpful/harmless/honest hard, Closed → Closed: GPT-4/Claude/Gemini → powerful closed API expensive business → vs vs open Source: Llama/Mistral
- Agent: LLM with reasoning + planning + tool use + activators for autonomous lang; Thought-Action-Reflection frameworks vs LLM = text generation only, passive.

LLM HANDBOOK

19/20

50 Must-Know Concepts

19

SECTION NAME: LLM CAREER & SALARY - ROADMAP

★ What is LLM Engineer?

Engineer who builds applications using LLMs – RAG, agents, fine-tuning.

Skills Needed:

"Python"

LLMs Transformers,

RAG Vector DBs Chroma Pinecone,

LangChain

LlamaIndex,

Prompt Engineering.

Fine-tuning

LLoRA

Agents

Deployment vLLM

Roadmap to LLM Engineer:

Step 1 Python Basics – 1 month

Step 2 ML Basics – 1 month

Step 3 Deep Learning Transformers – 1 month

Step 4 LLM Fundamentals – Tokenization Embeddings
1 month

Step 5 Prompt Engineering RAG Vector DBs – 1 month

Step 6 Fine-tuning LoRA QLoRA RLHF – 1 month

Step 7 Agents LangChain LangGraph – 1 month

Step 8 Projects Portfolio – 6 projects: RAG
Chatbot Agent Fine-tune SaaS YouTube Chat –
– 2 months

Total 8-10 months – zero to LLM Engineer.

Salary:

USA: Entry 120k-160k | Mid 160k-220k | Senior
220k-350k+

| FAANG 300k-600k+

India: Entry 12-20 LPA | Mid 20-40 LPA |
Senior 40-80 LPA | Top 80-150 LPA |
remote global 80k-200k USD.

LEVEL	USA	INDIA	SKILLS
Entry 0-2 years	120-160k USD	12-20 LPA	Python LLMs RAG LangChain
Mid 2-5 years	160-220k USD	20-40 LPA	Prompt DoRA Agents
Senior 5+ years	220-350k+ USD	40-80 LPA	System Design Scaling Research

Career Path:

Junior LLM Engineer → LLM Engineer → Senior LLM Engineer →
Staff LLM / AI Architect → Head of AI / CTO

How to get first job? Build 6 projects portfolio
needs LLM Engineer – 50k+ jobs [Linkamlit](#) /
Hugging Face Spaces deployed live → [LinkedIn posts explain](#)
LinkedIn posts explain projects → [apply jobs as engineer](#)
apply LLM Engineer roles.

Key Terms –

LLM Engineer: builds LLM apps using RAG, agents, fine-tuning.
Skills: Python, LLMs/Transformers, RAG/VectorDBs Chroma/Pinecone,
LangChain/LlamaIndex, Prompt Eng, LoRA/QLoRA/RLHF, Agents,
Deployment vLLM. Roadmap: 8-10 months – Steps 1-8 Python→ML→DL/
Transformers→LLM Fundamentals→Prompt+RAG→Fine-tune LoRA/QLoRA/RLHF→Agents
Agents→Projects Portfolio = Total 8-10 months. Salary: USA 120-350k+ FAANG
300-600k. India 12-80 LPA Top 80-150 LPA. Remote 80-200k USD.
Entry 0-2y 120-160k / 12-20LPA, Mid 2-5y 160-220k / 20-40LPA,
Senior 5+y 220-350k+ / 40-80LPA. Path: Junior → Engineer → Senior →
Staff/AI Architect → Head of AI/CTO
Demand: High 2024-2026, 50k+ jobs, future-proof. First job: 6
projects GitHub README, Streamlit/HF Spaces live, LinkedIn posts.

LLM HANDBOOK

20/20

50 Must-Know Concepts

20

SECTION NAME: FINAL RECAP

✓ Topic vs Status — Completed — all ✓

Topic	Status	Completed
1. What is LLM?	Large Language Model • billions params	✓
2. Scale Parameters	7B/13B/70B/175B/1.7T timeline	✓
3. Transformer Architecture	Attention • QKV • Multi-Head	✓
4. Tokenization & Embeddings	BPE = 50k vocab • 128k context	✓
5. Pre-training	CommonCrawl • Next Token • Scaling Laws	✓
6. Fine-tuning	SFT + PEFT + LoRA/QLoRA 4-bit	✓
7. RLHF Alignment	Zero/Few-shot • CoT • Temp/Top-p	✓
8. Prompt Engineering	Retrieval+Generation • vectorDB	✓
10. Vector DBs	Chroma/Pinecone/FAISS • LangGraph	✓
13. Optimization	Quantization • KV Cache • FlashAttn	✓
14. Security & Safety	LangChain • LlamaIndex • vLLM • HF	✓
15. Trends Future Tools	vLLM • PagedAttention • MoE • Agents	✓
20. Career Roadmap	Salary • Skills • Portfolio • 120k-600k	✓

20 SLIDES — FINAL RECAP (CONDENSED CHECKLIST)

- What is LLM? — LLM=Large Language Model • Deep Learning • trained on massive text • billions parameters • predicts next token • examples: GPT-4, Claude, Gemini, Llama, Mistral • emergent abilities • zero-shot & few-shot • vs Traditional: feature eng vs pretrained foundation model
- Scale Parameters — 7B / 13B / 70B / 175B / 1.7T params • Data: trillions tokens • Compute: thousands GPUs • Timeline: GPT-1 2018 117M 2019 1 GPT-2 3.4T — EF — 2020 → GPT-3 175B → GPT-4 2023 1.7T
- Transformer Architecture — Attention Is All You Need 2017 • Encoder-Decoder • Self-Attn • Tokenizing → Embedding → "tokens/subwords" → BPE 50k vocab • Embedding: - vector arithmetic king+man+woman=queen • Dim: 768/12288 • Multi-Head 12 heads Q&K&V • Attention Formula: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$
- Tokenization & Embeddings • Massive corpus: Common Crawl 60% • Wikipedia 3% • Books 15% • Code • Objective: Next Token Prediction • Gradient Descent • Base Model • Cost: \$4M-\$100M • Scaling Laws: Loss $\propto N^{-0.17}$ • Chinchilla 70B + 1.4T tokens > 280B better
- Fine-tuning — Instruction Tuning • Full vs PEFT • LoRA/QLoRA • rank 8/16 only • W&BA • 4-bit quantized • 65B model on 48GB GPU • 0.7=balanced • Top-p • 3) RL w/ PPO $\approx$ KL penalty • 3H: helpful+harmless+honest • DPO / -logit-BM
- Prompt Engineering • Zero-Shot • Full vs PEFT • LoRA/QLoRA • rank 8/16 • only 0.1% params • W&BA • 4-bit quantized • 65B model on 48GB GPU • 3 Steps: 1) SFT 2) REFT 3) PPO • KL peccore (mangled) • Weaviate • Hallucination • Flow: Docs → Chunk 500tokens → semantic 500-1000 • Index: HNSW/ANN
- RAG • Perplexity ↓ = Accuracy • F1 • BLEU • ROUGE • Benchmarks: MMLU 57.5 GPT-4 86% • GSM8 • GSM8K • HumanEval • HellaSwag • TruthfulQA • BIG-Bench • OutArise • Contamination & Hallucination • Reduce halluc. RAG-as-a-Service • LLM+Tools+Memory+Frontend+UI/UX/AutoGPT/BabyAGI/Oswal • Function Calling
- Security & Safety • LangChain: chains/agents/RAG • LlamaIndex: data ingestion+RAG • LangGraph • Semantic Kernel • Haystack • Guidance • Opinion Calling AI SaaS Next.js+Stripe+Chat • Transformers transcript local • LLM Studio • vLLM • TGI • Components: Models/Prompts/Chains/Agents/Memory/Retrievers/RAG chunking Chain vs OpenAI Limbs/R chain
- Optimization • RAG Chatbot on PDFs w/ Chroma+Streamlit 1M=10M • Small SLMs: Phi-3 3.8B • MoE: Mistral 1.8T/7.8B & experts 2 active • 2-10 moe6.0 • Aggrs/AuGPT • G6I 2027-2036 • SLMs MUST KNOW: Hugging Face World Model • Embedded AI: Challenges: Hallucinations Bias • Cloud vs Self-hosted vs Edge Privacy: Alignment • Closed vs Open: Llama/Mistral • jops

No.	Question
1	What is Transformer?
2	What is Self-Attention?
3	What is RAG?
4	Fine-tuning vs RAG?
5	What is LoRA / QLoRA?
6	What is RLHF? RLHF
7	Prompt Engineering?
8	Hallucination?
9	Quantization?
10	What is Agent?
11	What is Tokenization?
12	What is Embedding?
13	What is Scaling Laws?
14	What is vLLM?
15	What is MoE?

Save this guide • Share with team • Interview Tip: Build RAG • Chatbot • Agent • Fine-tune projects • hands-on ☆

Comment "PDF" for full guide

